

《《《 [返回首页](#)

《《《 [上一节](#)

循环

在这里，我们的代码也是计算整数列表的总和

```
List<Integer> ints = Arrays.asList(1,2,3);
int s = 0;
for (int n : ints) {
    s += n;
}
assert s == 6;
```

第三行中的循环被称为 `foreach` 循环，即使它是用关键字。它相当于以下内容：

```
for (Iterator<Integer> it = ints.iterator(); it.hasNext(); ) {
    int n = it.next();
    s += n;
}
```

强调的代码对应于用户编写的内容，编码器以系统的方式添加代码。它引入了 `Iterator<Integer>` 类型的变量来迭代 `List<Integer>` 类型的列表。通常，编译器会创建一个新的名字，保证不会与代码中已有的名字冲突。请注意，当 `Integer` 类型的表达式 `it.next()` 被分配给 `int` 类型的变量 `n` 时，发生拆箱。

`foreach` 循环可以应用于任何实现了接口 `Iterable<E>`（在包 `java.lang` 中）的对象，该对象继而引用接口 `Iterator<E>`（在包 `java.util` 中）。这些定义了 `iterator`，`hasNext` 和 `next`，它们被 `foreach` 循环的翻译使用（迭代器也有一个方法 `remove`，它不被翻译使用）：

```
interface Iterable<E> {
    public Iterator<E> iterator();
}
interface Iterator<E> {
    public boolean hasNext();
    public E next();
    public void remove();
}
```

集合框架中的所有集合，集合和列表都实现了 `Iterable<E>` 接口；而其他供应商或用户定义的类也可以实现它。`foreach` 循环也可以应用于一个数组：

```
public static int sumArray(int[] a) {
    int s = 0;
    for (int n : a) { s += n; }
```

```
        return s;
    }
```

`foreach` 循环是故意保持简单的，只捕获最常见的情况。如果你想使用 `remove` 方法或者并行迭代多个列表，你需要明确地引入一个迭代器。这是一个从 `Double` 列表中删除负值的方法：

```
public static void removeNegative(List<Double> v) {
    for (Iterator<Double> it = v.iterator(); it.hasNext();) {
        if (it.next() < 0) it.remove();
    }
}
```

这里是计算两个向量的点积的方法，表示为 `Double` 列表，两个长度都相同。给定两个向量: `u1, ... , un` 和 `v1, ... , vn` 他们计算： $u_1 * v_1 + \dots + u_n * v_n$:

```
public static double dot(List<Double> u, List<Double> v) {
    if (u.size() != v.size())
        throw new IllegalArgumentException("different sizes");
    double d = 0;
    Iterator<Double> uIt = u.iterator();
    Iterator<Double> vIt = v.iterator();
    while (uIt.hasNext()) {
        assert uIt.hasNext() && vIt.hasNext();
        d += uIt.next() * vIt.next();
    }
    assert !uIt.hasNext() && !vIt.hasNext();
    return d;
}
```

两个迭代器 `uIt` 和 `vIt` 在锁定步骤中跨越列表 `u` 和 `v`。循环条件只检查第一个迭代器，但断言确认我们可以有使用第二个迭代器，因为我们先前测试了两个列表来确认他们有相同的长度。

《《《 [下一节](#)

《《《 [返回首页](#)